NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

SC2006 – Software Engineering

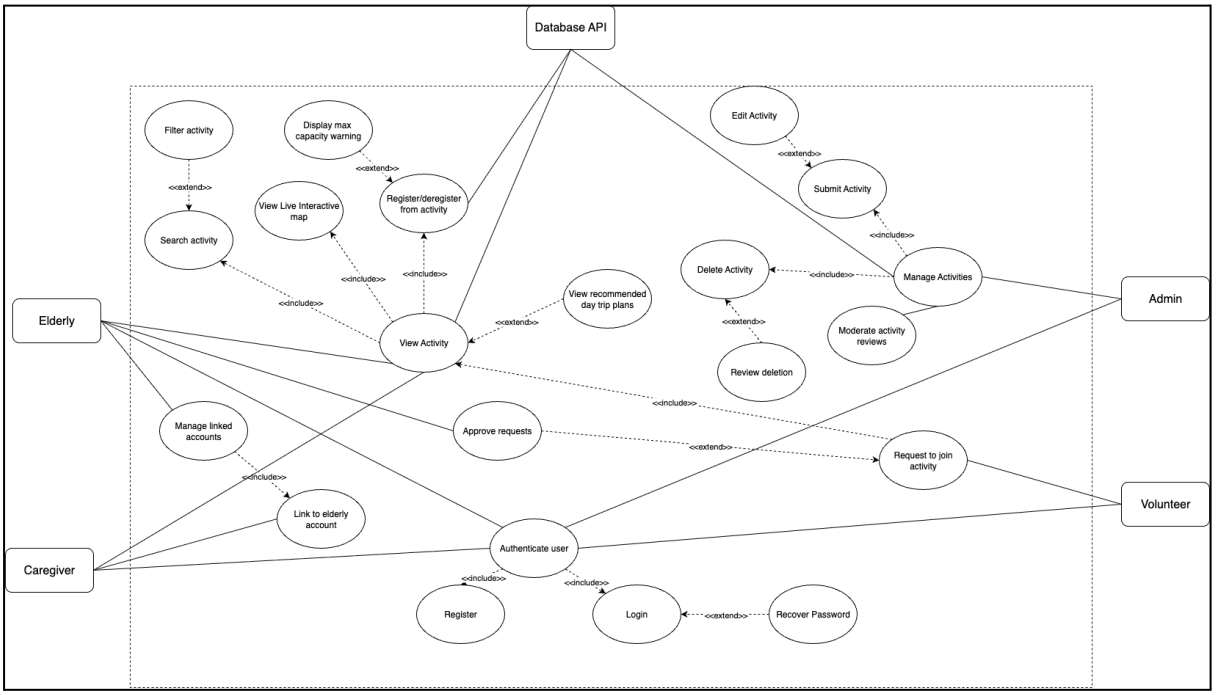
Lab 3 Deliverables

Lab Group	SCE3
Team	SilverConnect
	Lucas Cheong Wai (U2421947B)
	Wu Songxin
Members	Nicodemus Noel Lee
	Sivakumar Magina
	Lim Jia Yi

Table of Contents

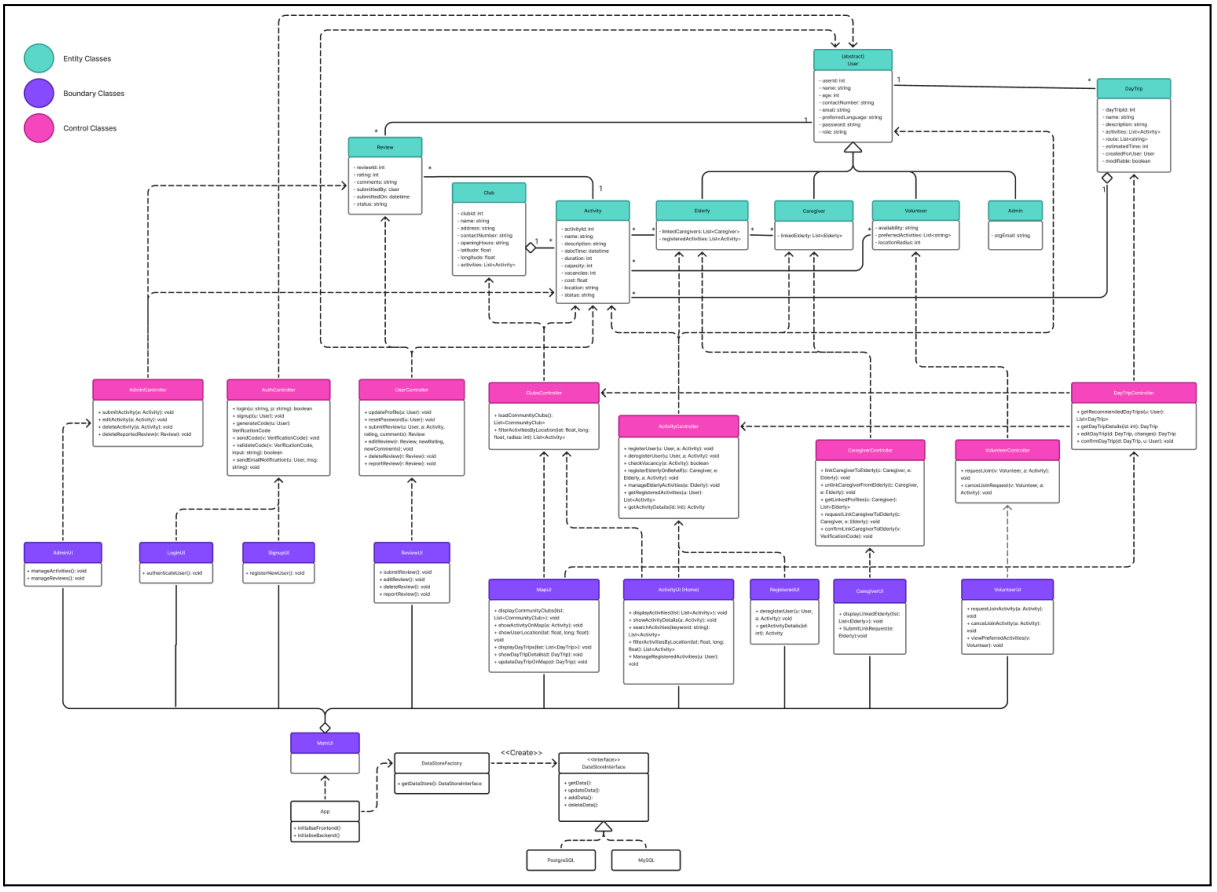
1. Complete Use Case Model **3**
2. Design Model **4**
A. Class Diagram 4
B. Sequence Diagrams 10
I. For Functional Requirement #1 10
I.I ViewActivities 10
I.II RegisterActivities 11
I.III ViewLiveMap 12
I.IV ViewRecommenededDayTrips 13
II. For Functional Requirement #2 14
II.I LinkCaregiversToElderlyProfile 14
II.II CaregiversRegisterActivities 14
II.III CaregiversRegisterWithElderly 15
III. For Functional Requirement #3 16
III.I SubmitActivites 16
III.II EditActivity 17
III.III DeleteActivity 18
III.IV ManageActivity 19
III.V ModerateActivityFeedback 20
IV. For Functional Requirement #4 21
IV.I JoinActivity 21
IV.II ProvideFeedback 22
V. For Functional Requirement #5 23
V.I AuthenthicateUser 23
V.II Register 23
V.III Login 24
V.IV ForgotPassword 25
C. Dialog Map Diagram 25
3. System Architecture **26**
A. Presentation Layer 26
B. App Logic Layer 27
C. App Object Layer 28
D. Persistent Data Layer 29
4. Application Skeleton **30**
A. Frontend (React.js / TypeScript / Tailwind CSS) 30
B. Backend (Node.js / Express / TypeScript) 31
5. Appendix **33**
A. Key Design Issues 33
I. Identifying and Storing Persistent Data 33
II. Providing Access Control 33
B. Tech Stack 34

1. Complete Use Case Model



2. Design Model

A. Class Diagram



<u>App</u>

Start-up class responsible for the initialization of both the frontend and backend components of the SilverConnect system.

Operations:

- **initialiseFrontend():** Renders the web-based User Interfaces (UI) through the React frontend (e.g., LoginUI, MapUI, ActivityUI).
- **initialiseBackend():** Starts the Express backend server and calls getDataStore() from the DataStoreFactory for database access and manipulation through PostgreSQL/mySQL

<u>User Interfaces (UI)</u>

User Interfaces are what the users see and interact with on the application.

- **MainUI:** Acts as the entry point and navigation hub of the SilverConnect system. It connects all sub-UIs together and coordinates page routing after system initialization. Users can navigate between authentication, activity listings, and their registered activities etc.

- **LoginUI:** Handles user authentication. Allows users (Elderly, Volunteer, Caregiver, Admin) to log in using their registered credentials. Sends login requests to the AuthController for verification.
- **SignupUI:** Handles account registration for new users. Allows users to create a new account by providing personal details, role type, and password etc. Communicates with the AuthController to store new user data in PostgreSQL.
- **AdminUI:** Accessible only by administrators. Provides features for managing users, verifying volunteers, moderating reviews, and submitting, editing or deleting activities. Interfaces with the AdminController to retrieve or modify system data.
- **CaregiverUI:** Used by caregivers to link/unlink elderly profiles. Assuming the caregiver already knows the elderly's registered email on SilverConnect, he can link using that elderly's email and a link request will be sent directly to the elderly on SilverConnect where he approves/rejects. If approved, both the caregiver and elderly will be linked and stored in the elderly_caregivers table in postgres database.
- **VolunteerUI:** Allows volunteers to send join requests to the registered elderly for that activity. If the elderly accept their join request on SilverConnect, the volunteer will be automatically registered for that particular activity.
- **MapUI:** Displays an interactive map showing nearby community clubs. Helps users find the best routes to their registered activities at the respective community clubs. Additionally, there will be recommended day trip options that include their registered activities, hawker, and parks. Uses data from the MapController, which retrieves coordinates and directions and DayTripController to create day trip options tailored based on the elderly's interest.
- **RegisteredUI:** Displays all activities the user has registered for. Allows users to cancel a registration. Communicates with the ActivityController and the PostgreSQL database to update the registration list.
- **ActivityUI (Home):** The main home page that lists all available activities. Users can browse, filter, and join activities directly from this screen. Connects with the ActivityController for data retrieval.
- **ReviewUI:** Allows users to submit, view, and edit reviews for activities they participated in. Reviews can include ratings and feedback. Interacts with the UserController to store or update review data in the PostgreSQL database.

<u>Controllers (Facade Pattern)</u>

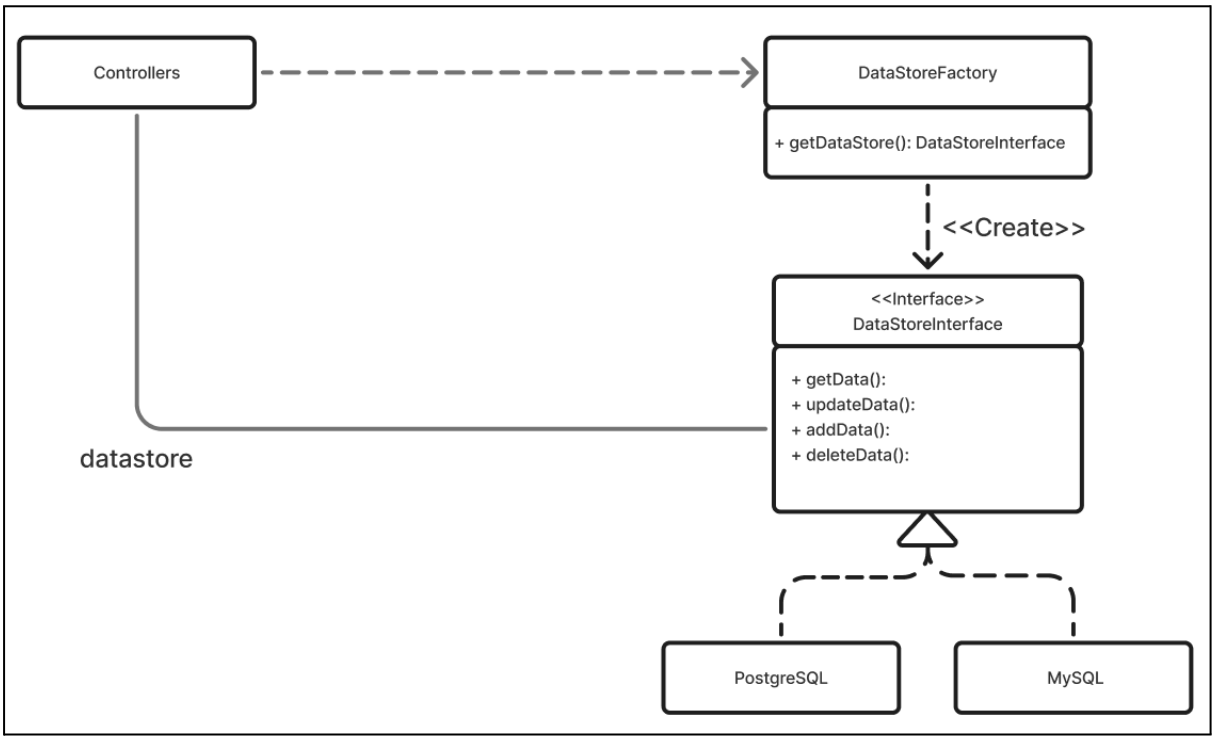
Application of the Facade Pattern, where controllers act as simplified interfaces that coordinate communication between the User Interfaces (UI) and the system's underlying business logic or services. Multiple controllers are implemented, each responsible for handling specific features or user roles within the SilverConnect system.

- **AuthController:** Manages all authentication and registration processes. It handles user login, logout, and signup requests, verifying credentials against the PostgreSQL database and ensuring secure password handling. This controller is accessed by the LoginUI and SignupUI, providing users with seamless authentication and account creation functionality.
- **AdminController:** Provides administrative functions that allow system administrators to manage users, activities, and reviews. Administrators can verify volunteers, approve or remove activities, and moderate user reviews. It interacts primarily with the User, Activity, and Review entities and is accessed through the AdminUI.

- **UserController:** Manages operations related to user data and profiles. It allows retrieval and modification of user information, such as personal details or linked profiles, and handles review submissions and updates. This controller communicates with the User and Review entities and supports both the ReviewUI and CaregiverUI.
- **ClubsController:** Handles location-based services by retrieving and processing geospatial data from data.gov.sg. It displays nearby community clubs and activity venues and calculates optimal routes for users. The controller also collaborates with the DayTripController to generate recommended day trips based on registered activities. It interacts with the Activity entity and external APIs and is accessed through the MapUI.
- **DayTripController:** Generates day trip recommendations tailored to the interests of elderly users. It combines information from activities, nearby hawkers, and parks to create complete itineraries that enhance user engagement. The controller accesses the Activity, User, and Review entities and works with the MapUI.
- **ActivityController:** Manages all activity-related operations. It retrieves available activities from the PostgreSQL database, allows users to register or cancel participation, and ensures that capacity limits are maintained. This controller accesses the Activity and User entities and interacts with the ActivityUI and RegisteredUI.
- **CaregiverController:** supports caregivers in managing their assigned elderly profiles. It allows caregivers to search for elderly users, send requests for linking, and unlink profiles when necessary. When a caregiver successfully links with an elderly user, a new record is inserted into the **elderly_caregivers** table to establish the relationship. Conversely, if the caregiver unlinks, the corresponding record is deleted from the same table. This ensures that the database remains synchronized with the current caregiver–elderly relationships. The controller accesses the User entities and is primarily used by the CaregiverUI.
- **VolunteerController:** Manages volunteer participation in community activities. It enables volunteers to browse, join, or cancel registrations and view assigned tasks or upcoming events. This controller interacts with the User and Activity entities and is accessed via the VolunteerUI.

Data Access Layer – DataStore (Factory Pattern + Strategy Pattern)

The DataStore ensures consistent and reliable data access, modification, and storage across the SilverConnect system. It serves as the main communication bridge between the controllers and the underlying databases (PostgreSQL or MySQL).



The DataStore implements 2 design patterns - **Factory Pattern** and **Strategy Pattern**.

Factory Pattern

The Factory Pattern is implemented through the DataStoreFactory, which provides a centralized mechanism for the application to obtain a database implementation (PostgreSQL or MySQL) as defined in db.ts.

- The factory determines which concrete data store to instantiate based on configuration or runtime parameters.
- Each implementation establishes its own database connection (e.g., using the pg library for PostgreSQL or the mysql2 library for MySQL) and executes SQL queries defined in schema.sql or controller logic.
- While query execution is managed by controllers, the factory ensures all database interactions use a single, consistent connection instance.
- This design promotes modularity, simplifies database migration or integration, and maintains a clean separation between data persistence and business logic.

Strategy Pattern

The Strategy Pattern abstracts database access logic and allows multiple interchangeable database implementations. It provides a uniform interface for controllers to interact with any

supported database (e.g., PostgreSQL or MySQL) without requiring changes to controller code.

This is achieved through the following components:

- **DataStoreInterface**

  Defines a standard set of data access methods such as getData(), updateData(), addData(), and deleteData(). All concrete database implementations must adhere to this contract, ensuring consistency across data operations.

- **PostgreSQLDataStore**

  A concrete implementation of DataStoreInterface optimized for PostgreSQL. It encapsulates connection setup and query execution using the pg library, ensuring stable and efficient data handling.

- **MySQLDataStore**

  Another concrete implementation of DataStoreInterface, tailored for MySQL. It handles connection management and query execution using the mysql2 library, following the same interface structure to maintain consistency with PostgreSQL.

- **DataStoreFactory**

  Responsible for creating and returning the appropriate DataStoreInterface implementation (PostgreSQL or MySQL) based on configuration. Adding new databases (e.g., MongoDB) requires only an additional implementation and a minor update in the factory logic.

- **Controllers**

  Controllers handle high-level business logic and depend on the DataStoreFactory to obtain a suitable data store instance. They interact only through the DataStoreInterface, keeping them decoupled from specific database technologies.

<u>Other Design Patterns</u>

- **Observer Pattern**

  Observer Pattern is used to facilitate notification of new activities join request (Subject) for volunteers (Observers) to accept or reject.

- **Facade Pattern**

  The Facade Pattern is implemented through the controller layer, where controllers such as AuthController, ActivityController, and MapController act as unified interfaces between the frontend and the backend. Instead of the user interfaces directly handling complex database queries or logic, the controllers simplify interactions by managing authentication, activity management, and data retrieval internally. This

pattern hides system complexity, promotes modularity, and provides a clear, easy-to-use interface for the frontend to communicate with the backend.

- **Chain of Responsibility Pattern**

  Used in the volunteer request approval flow, where a join request moves through multiple handlers. First the Elderly user reviews it, then the system validates and finally notifies the Volunteer. Each handler processes or passes the request along, ensuring a clear, step-by-step approval chain.

- **Command Pattern**

  Applied when users perform actions like joining, canceling, or approving activities. Each action can be represented as a command object, making it easy to log, undo, or re-execute operations later if needed.

- **State Pattern**

  Used for activity and user status transitions, such as an activity moving between Open, Full, and Completed, or a caregiver–elderly link progressing from Pending to Linked to Unlinked. Each state defines unique behaviors and allowed operations, keeping logic clear and modular.

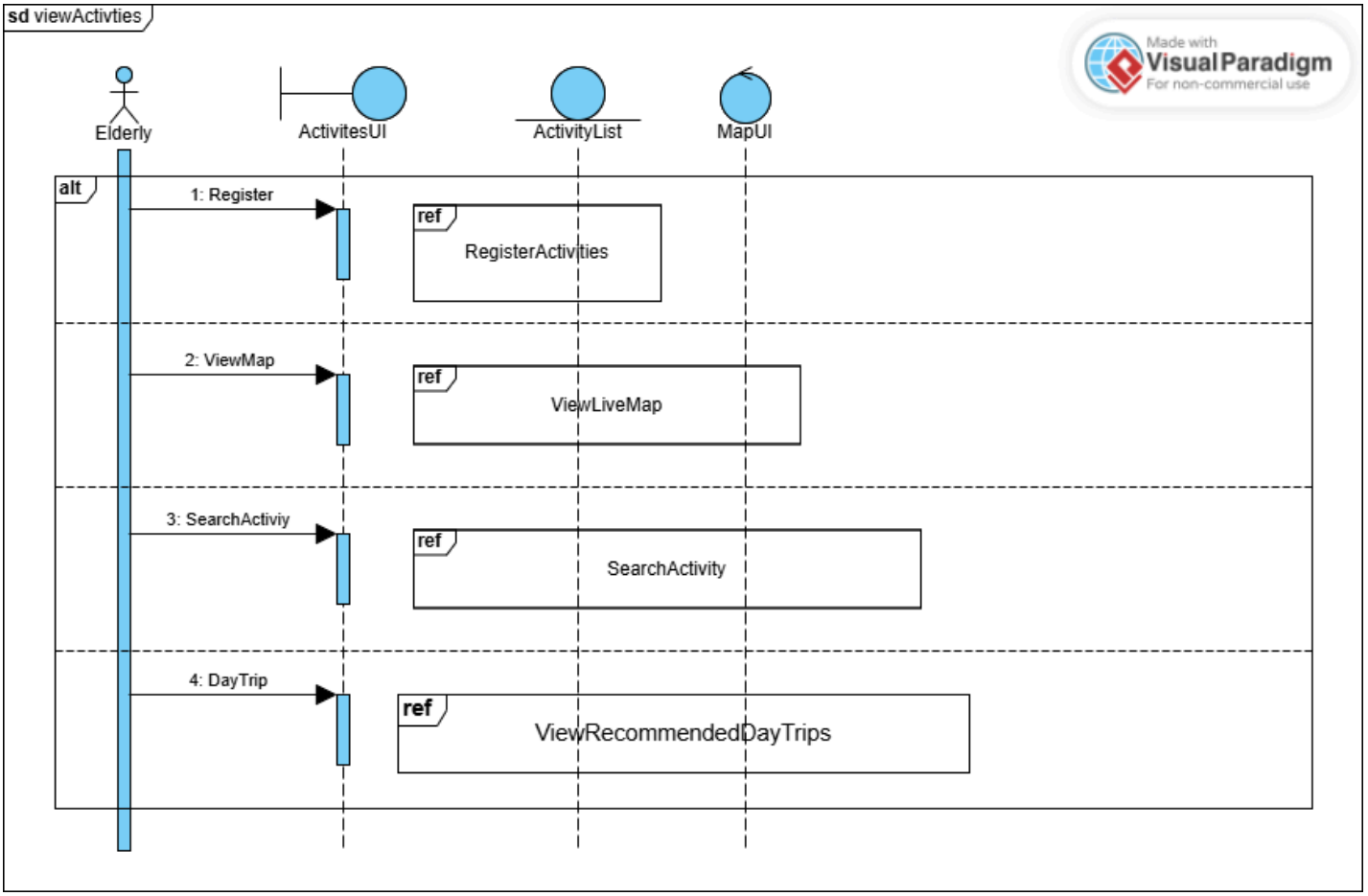
- **Publisher-Subscriber Pattern**

  Used for real-time messaging and notifications via WebSockets. When a user sends a message or triggers an event (like a link confirmation), the backend publishes it, and all relevant users (caregivers, volunteers, etc.) receive updates instantly through their UIs.

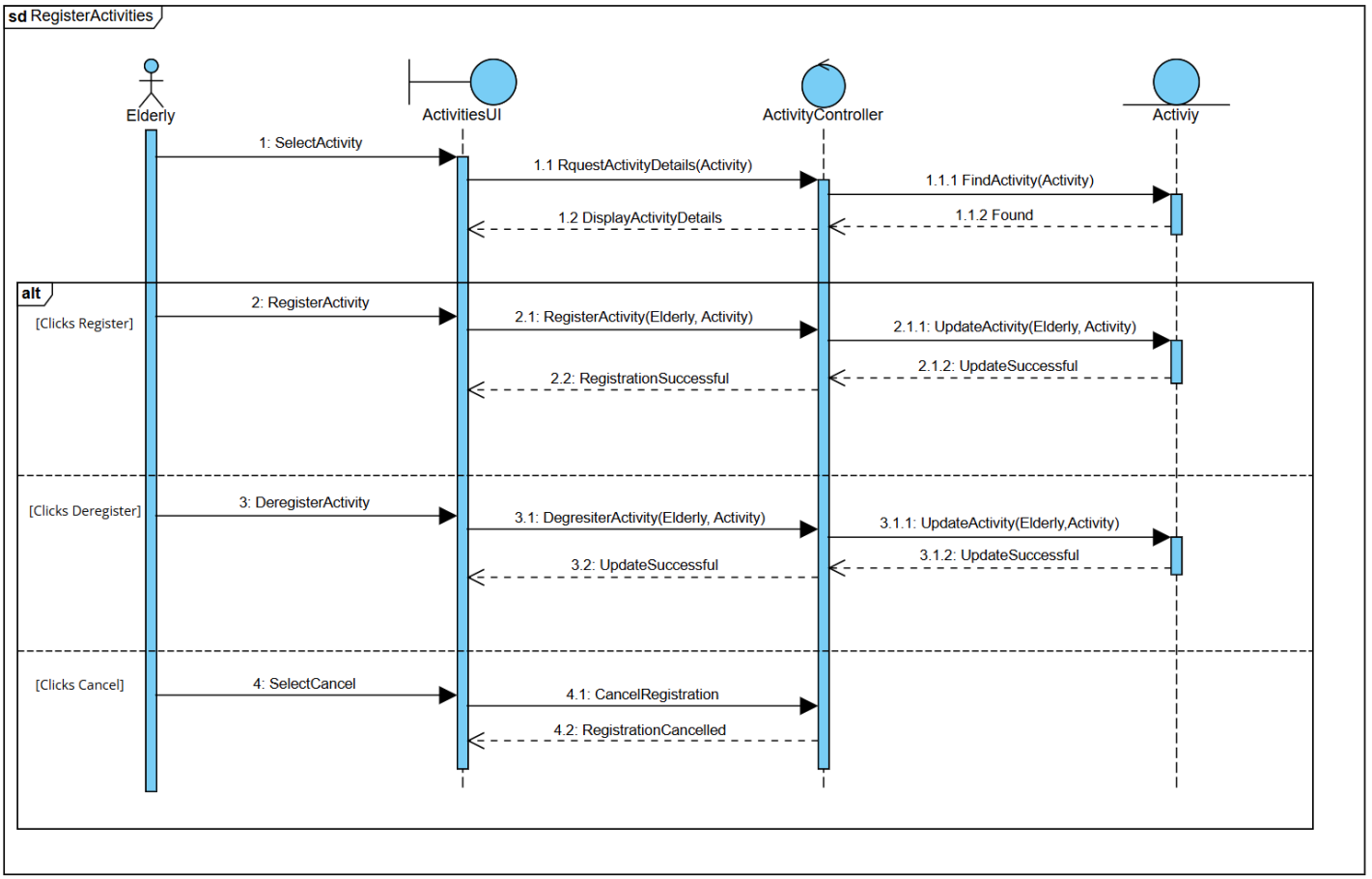
B. Sequence Diagrams

I. For Functional Requirement #1

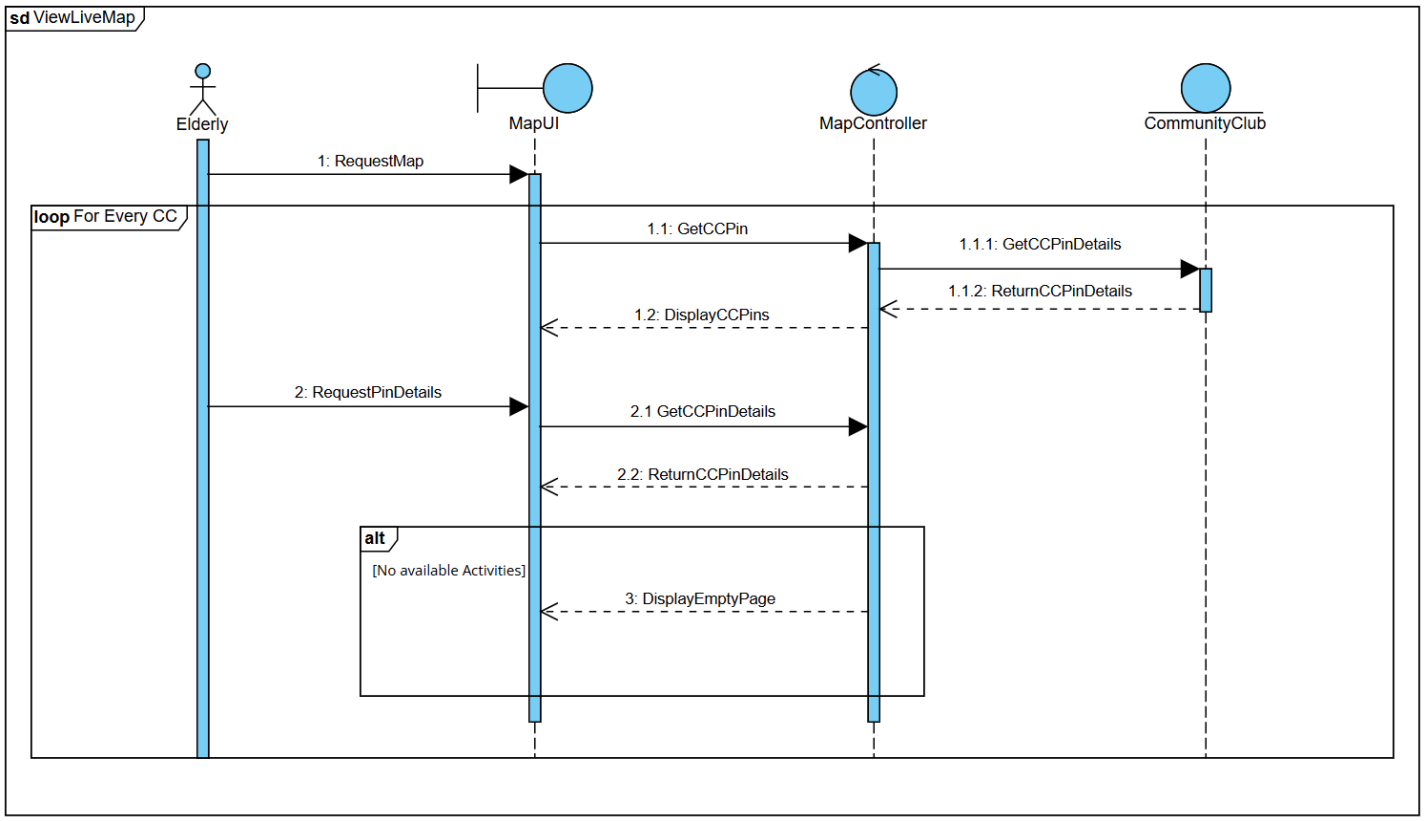
I.I ViewActivities



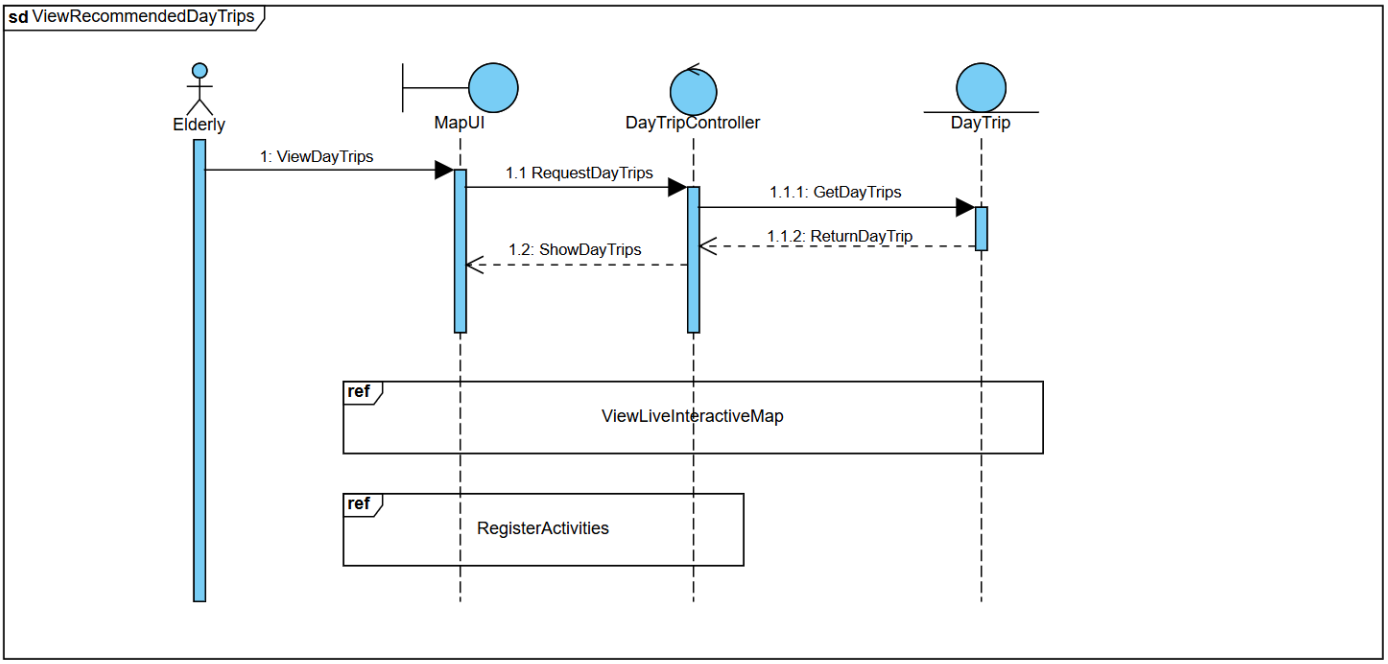
I.II RegisterActivities



I.III ViewLiveMap

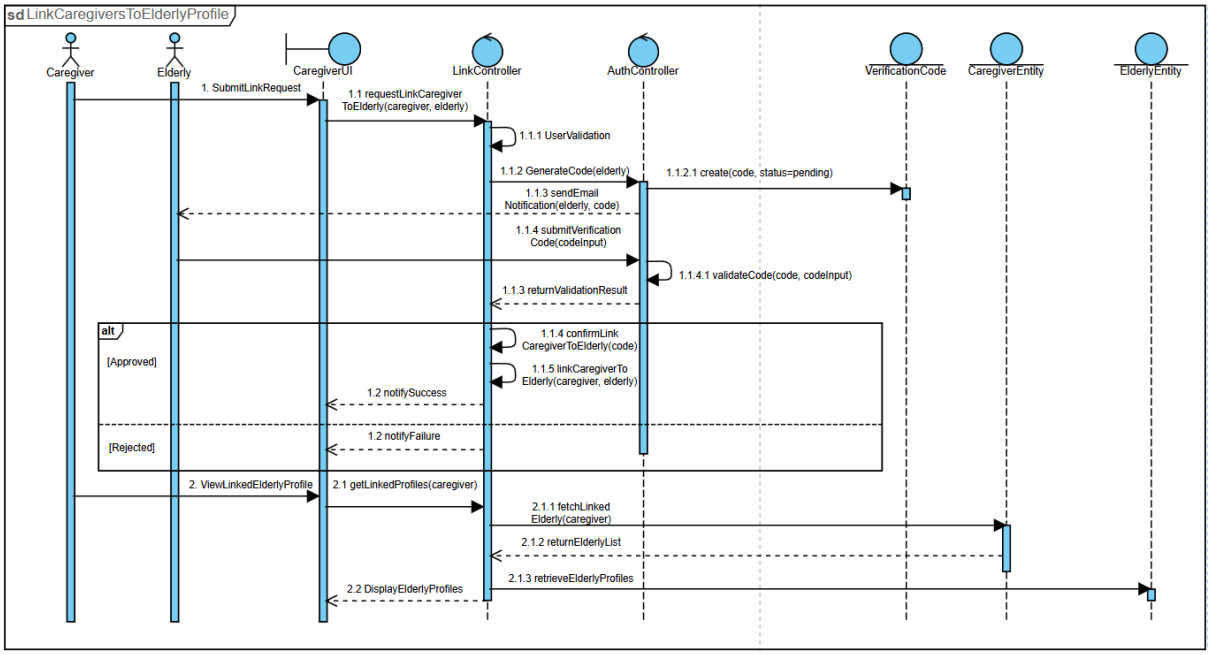


I.IV ViewRecommenededDayTrips

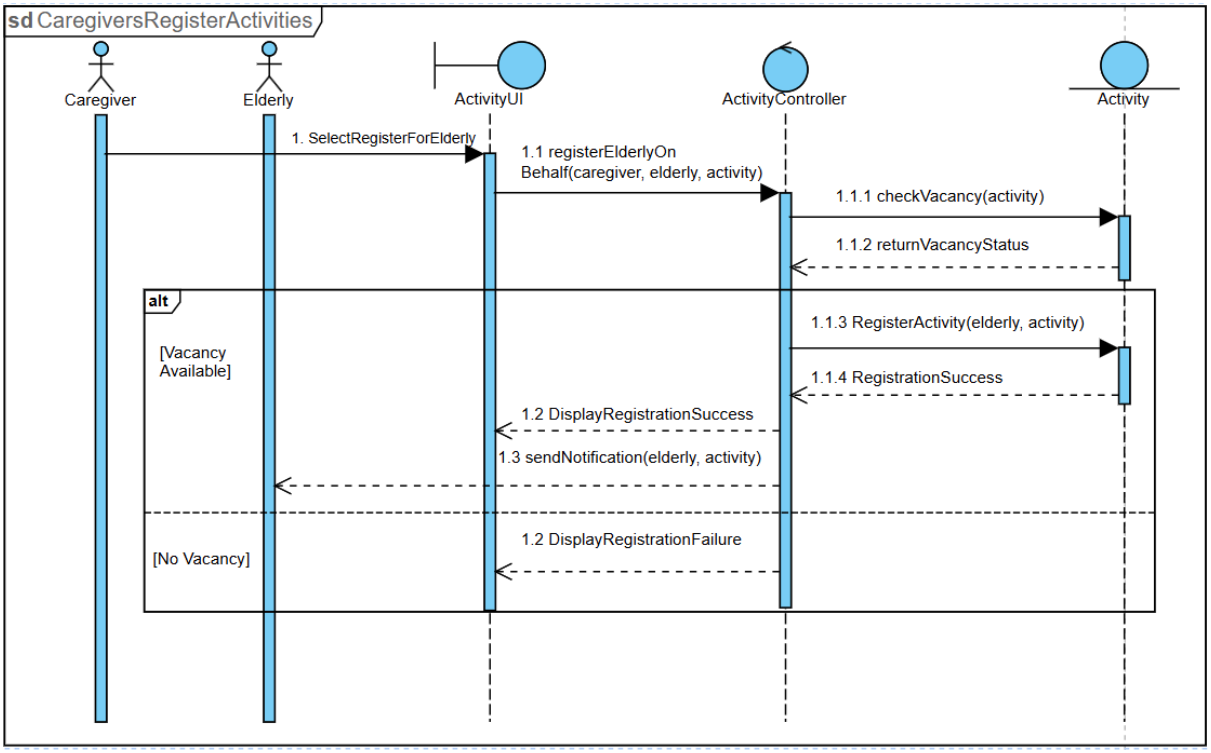


II. For Functional Requirement #2

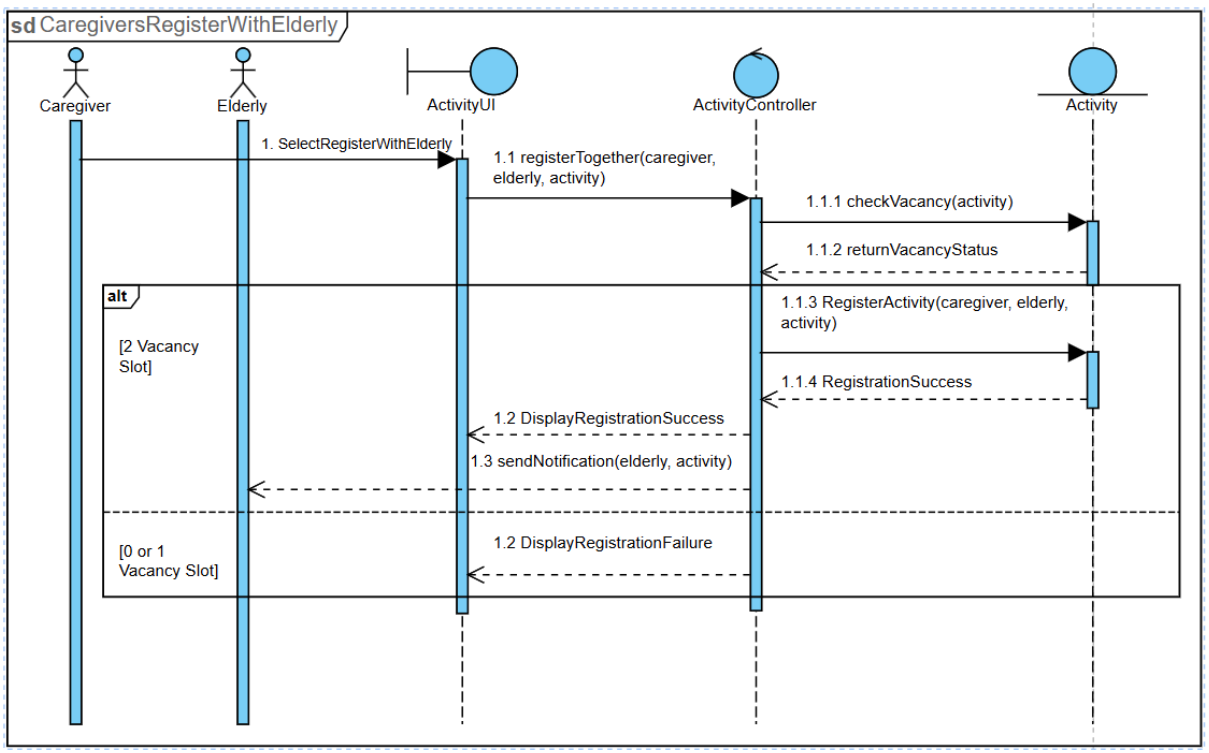
II.I LinkCaregiversToElderlyProfile



II.II CaregiversRegisterActivities

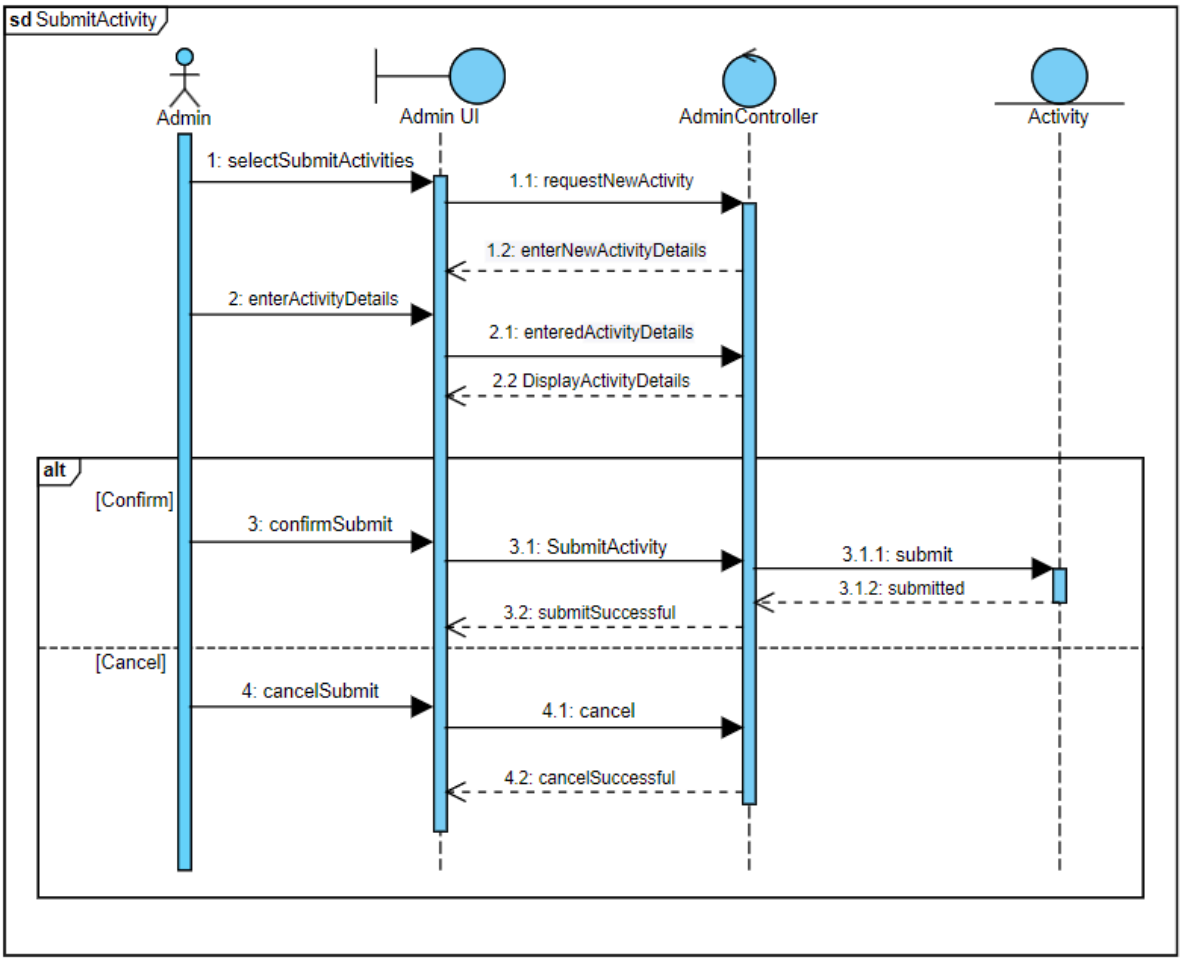


II.III CaregiversRegisterWithElderly

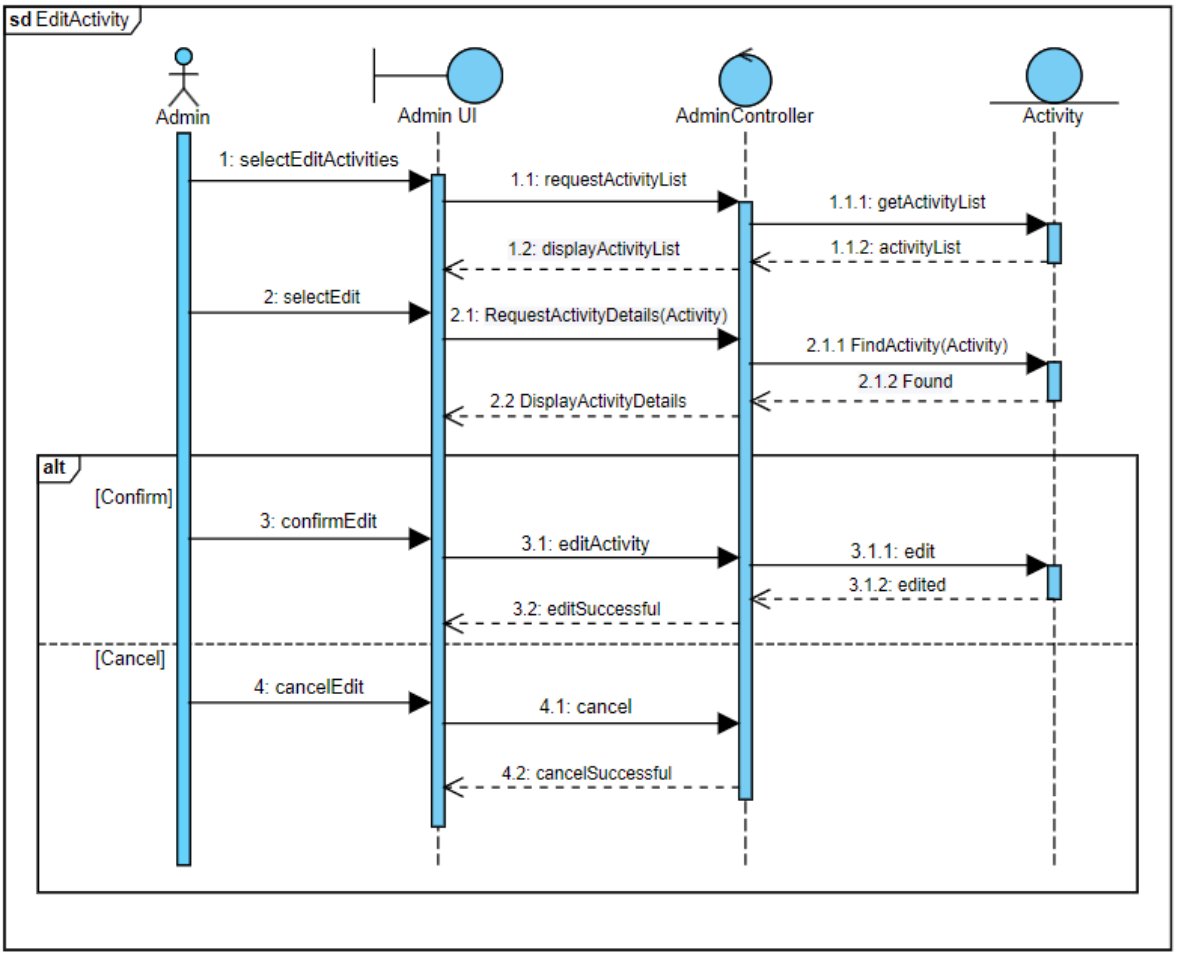


III. For Functional Requirement #3

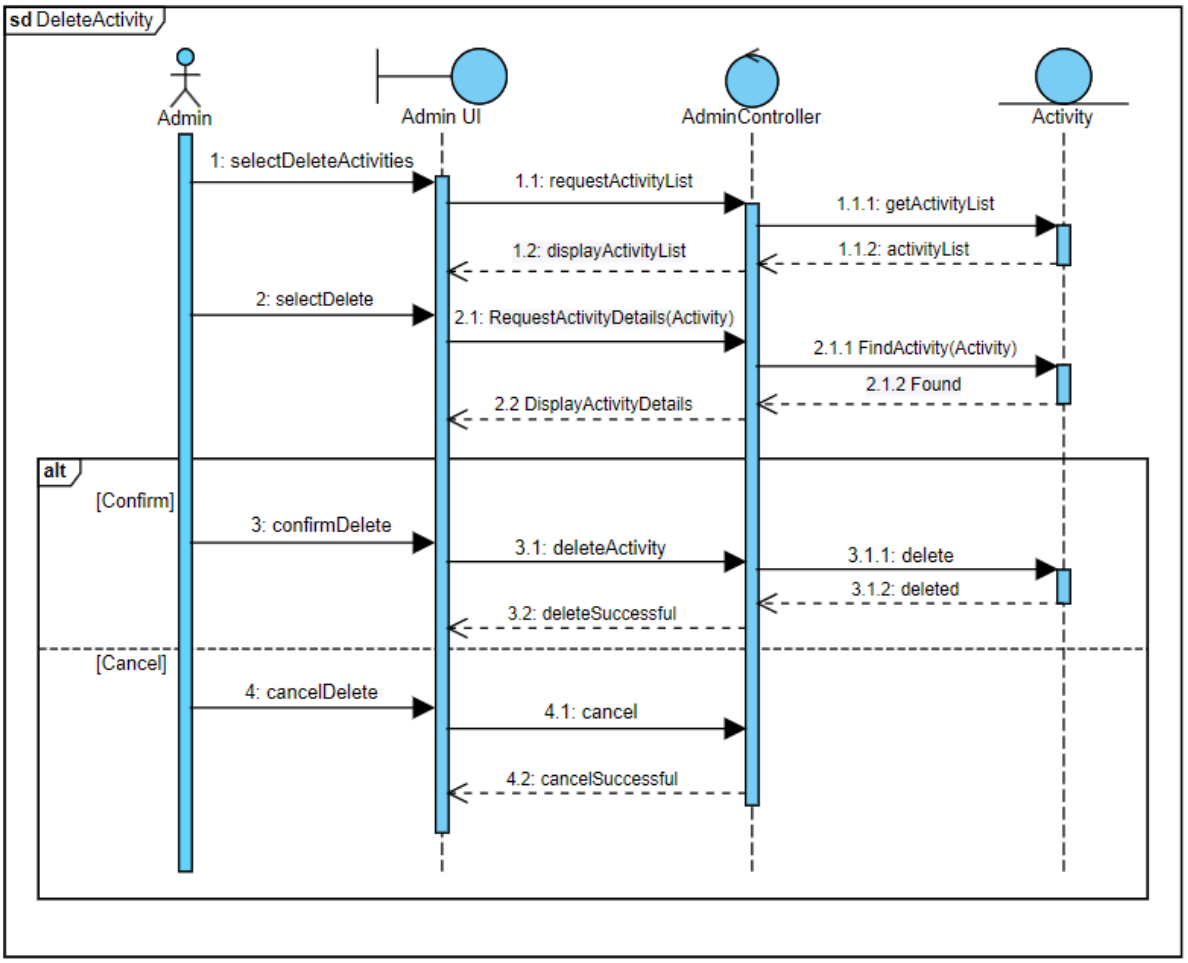
III.I SubmitActivites



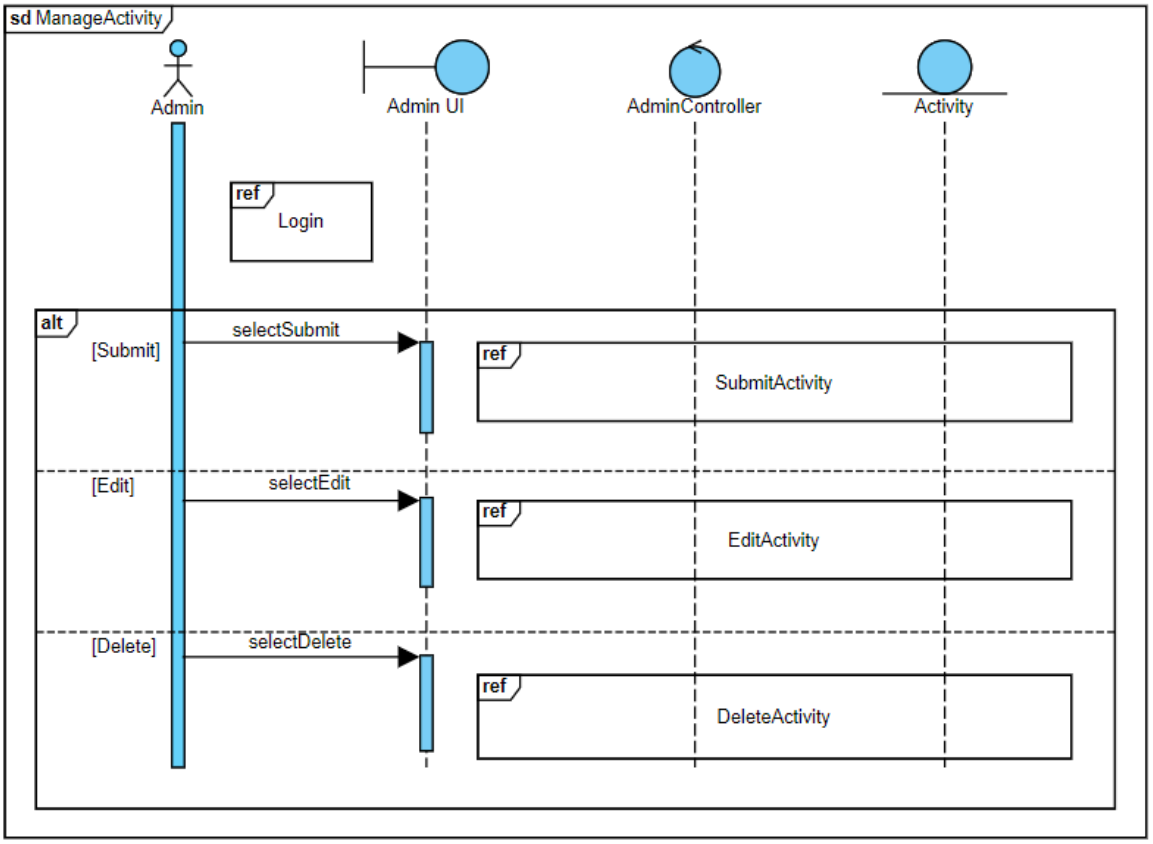
III.II EditActivity



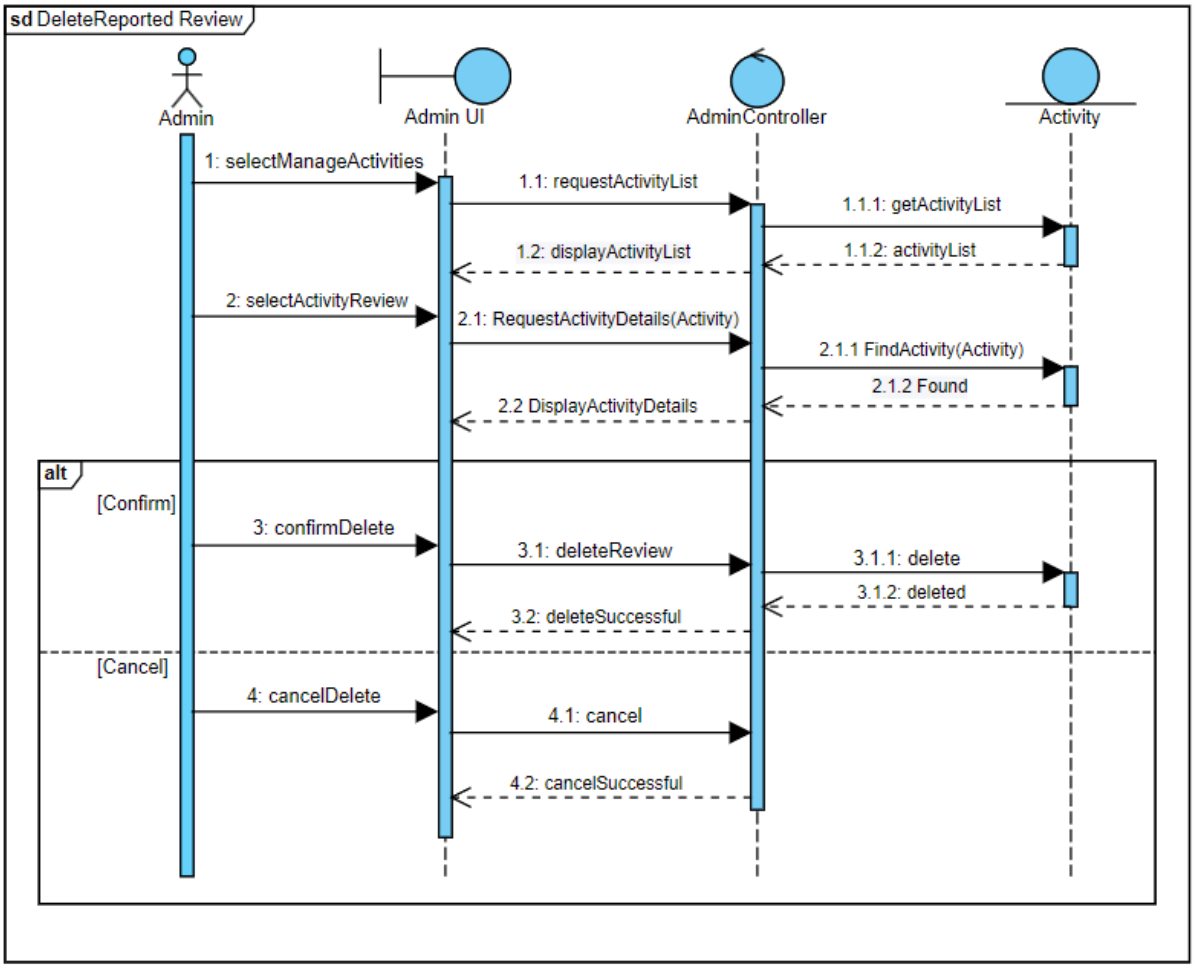
III.III DeleteActivity



III.IV ManageActivity

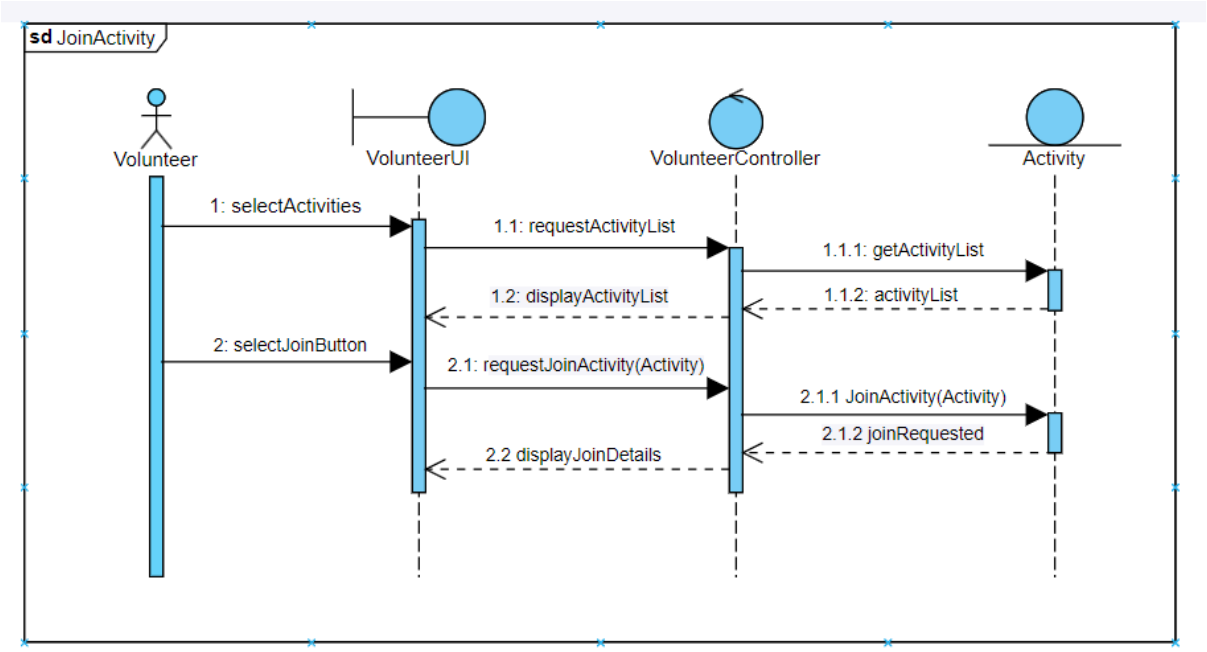


III.V ModerateActivityFeedback

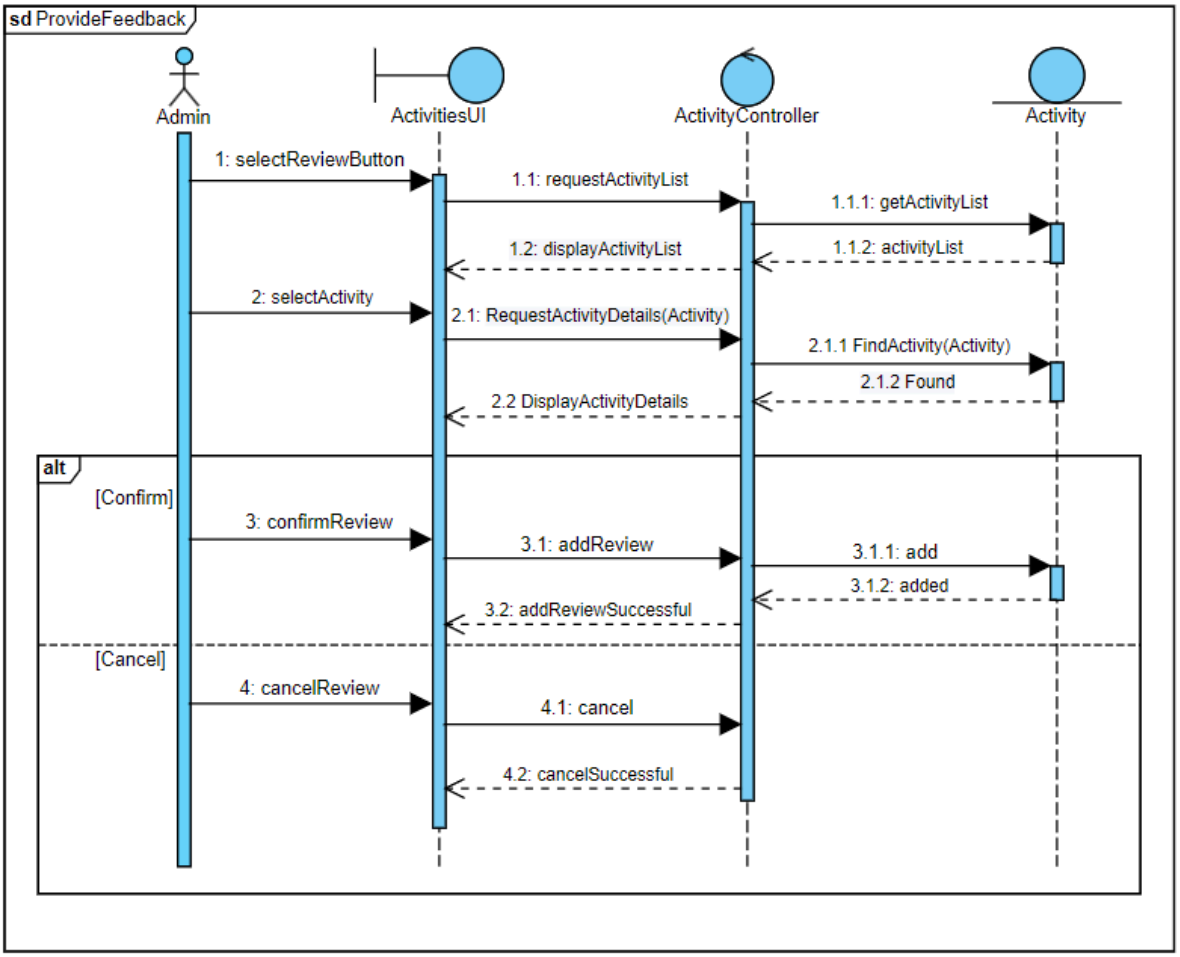


IV. For Functional Requirement #4

IV.I JoinActivity

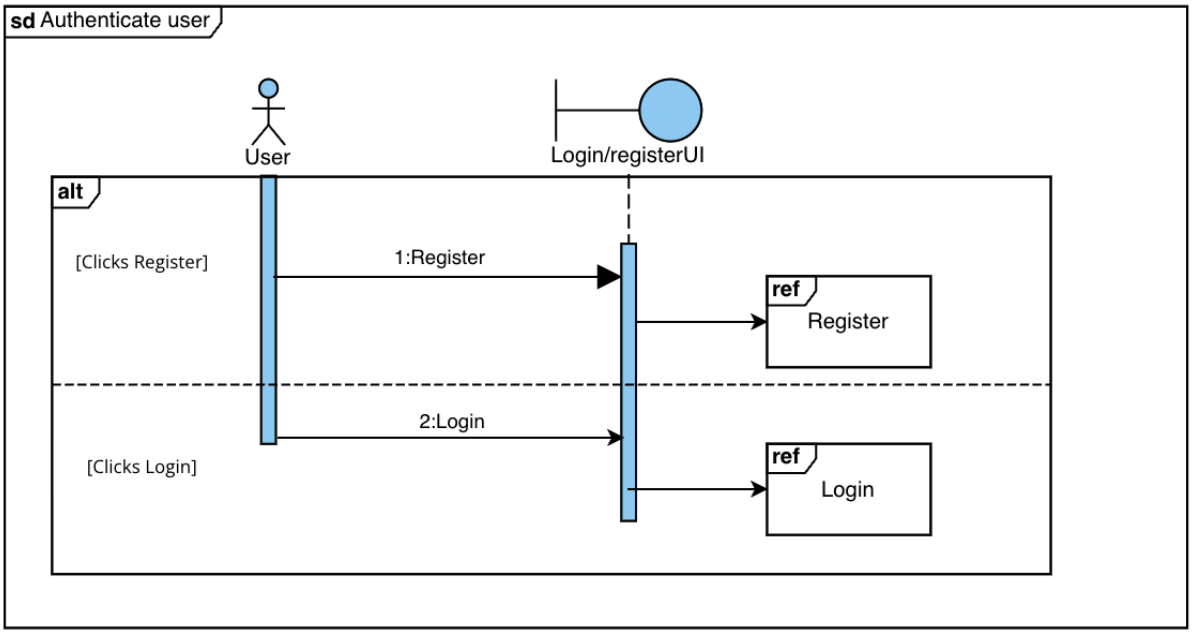


IV.II ProvideFeedback

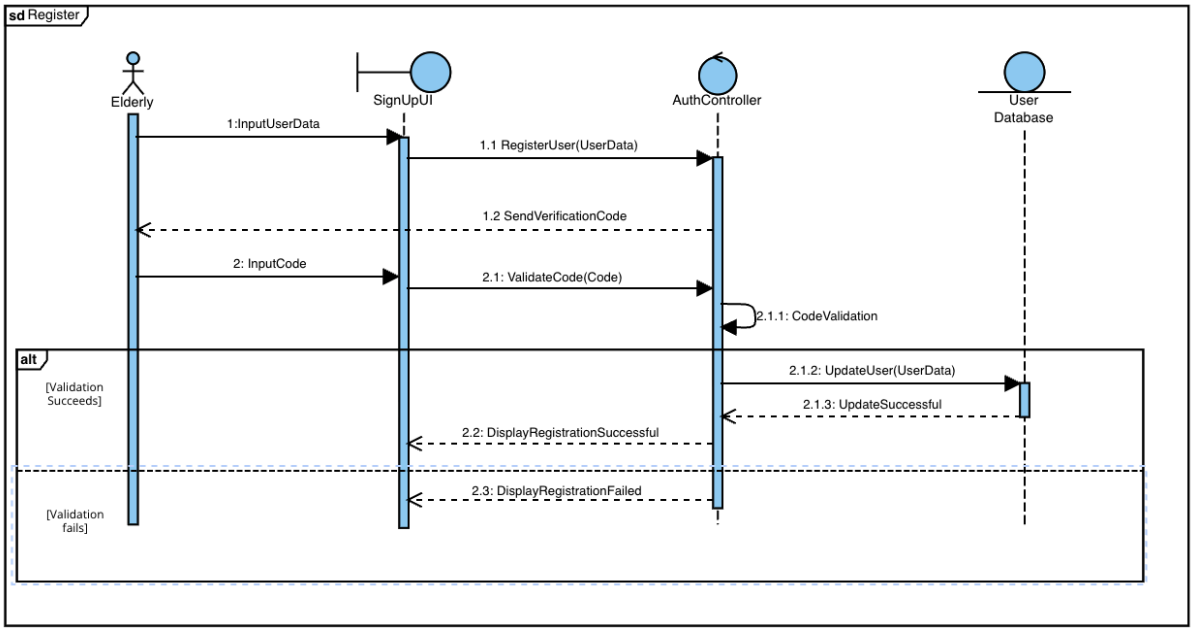


V. For Functional Requirement #5

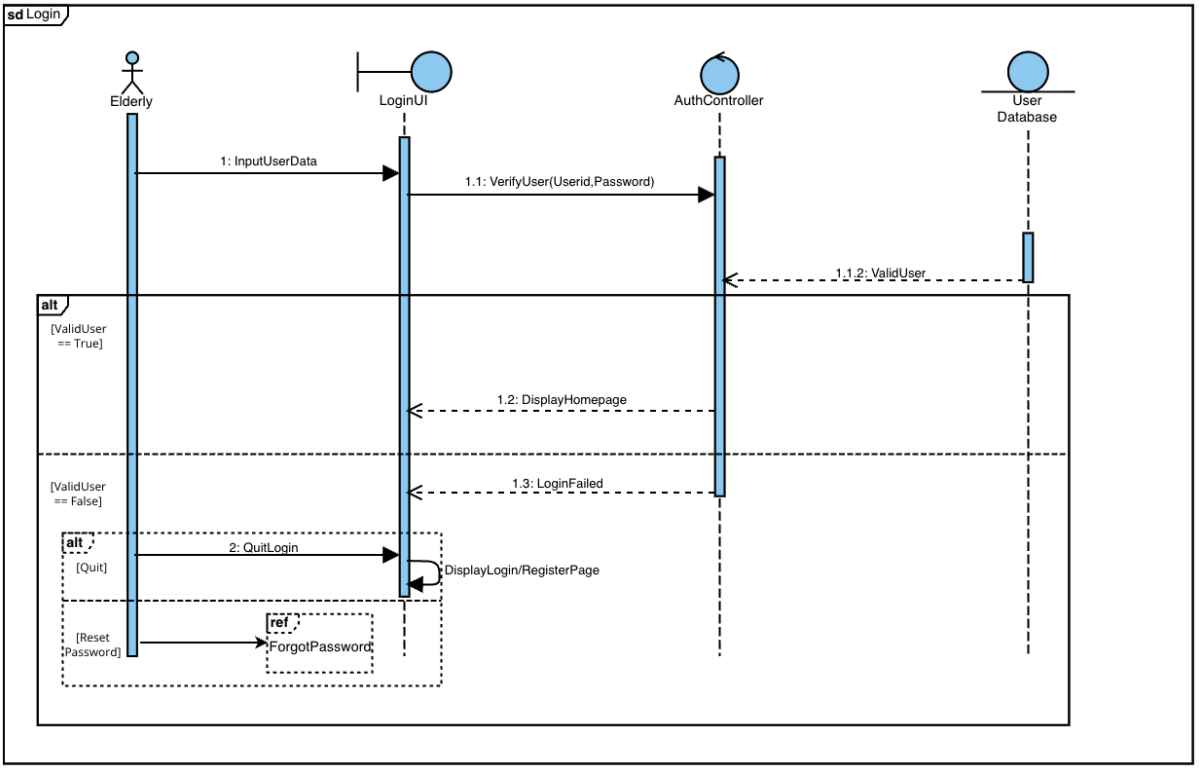
V.I AuthenthicateUser



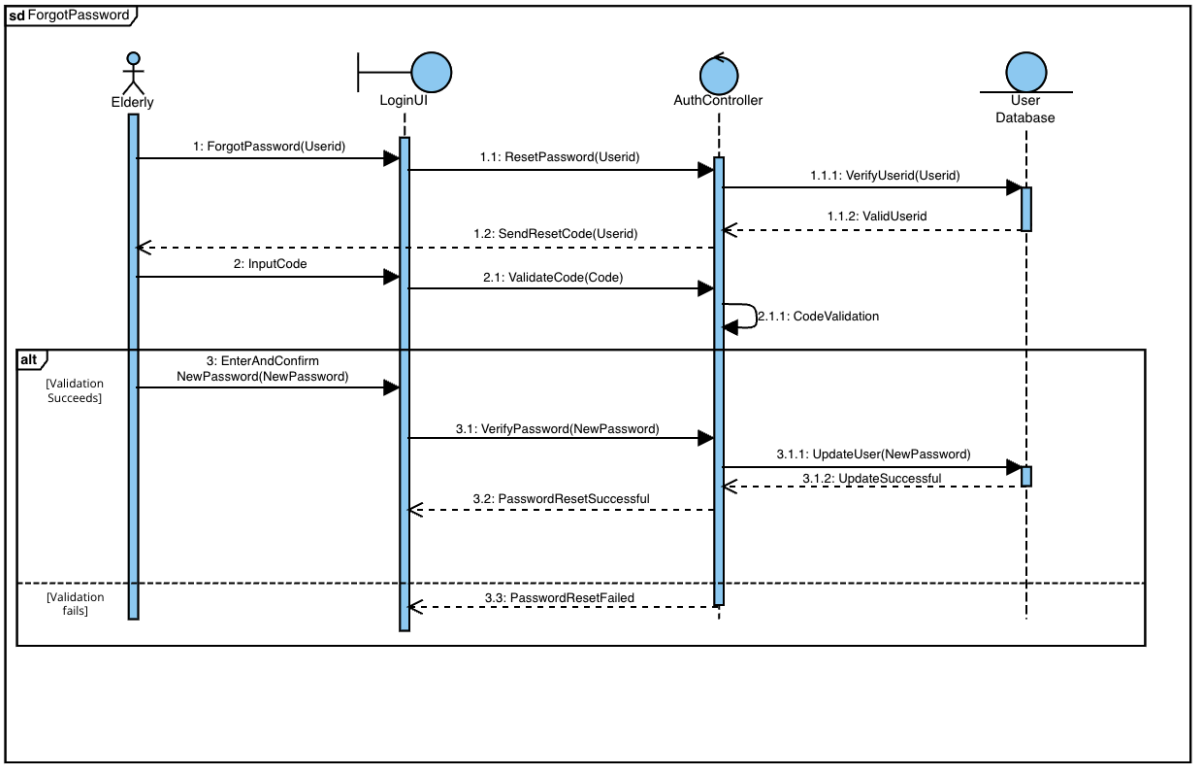
V.II Register



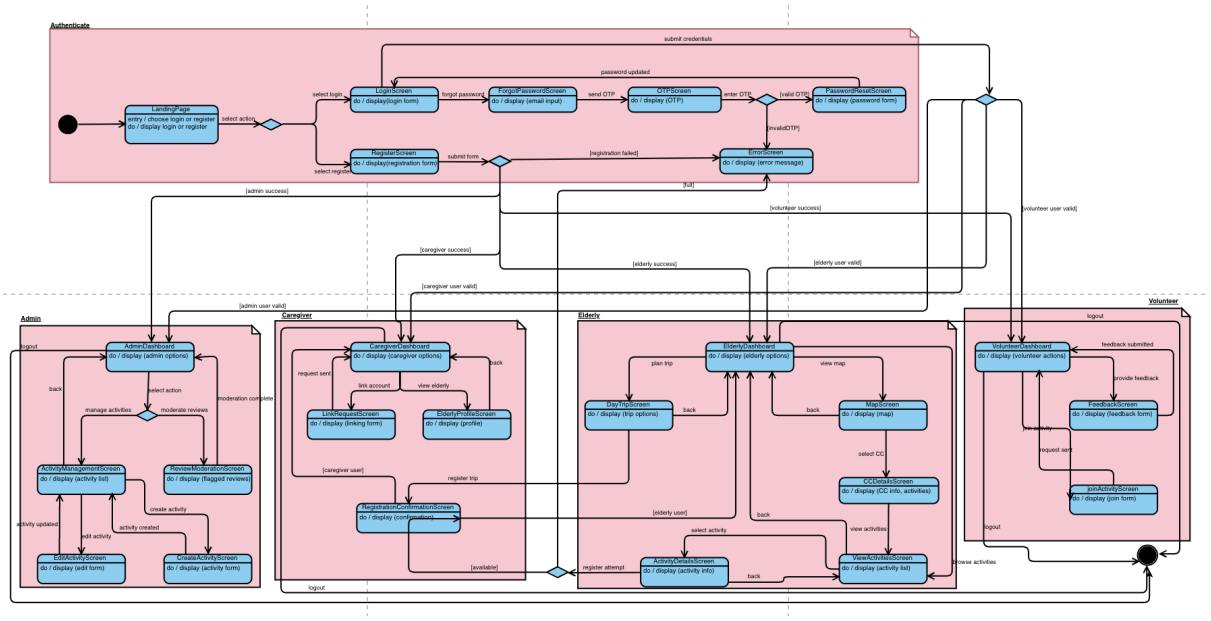
V.III Login


sd Login
Elderly
LoginUI
AuthController
User Database
1: InputUserData
1.1: VerifyUser(Userid,Password)
1.1.2: ValidUser
alt
[ValidUser == True]
1.2: DisplayHomepage
[ValidUser == False]
1.3: LoginFailed
alt
[Quit]
2: QuitLogin
DisplayLogin/RegisterPage
[Reset Password]
ref
ForgotPassword

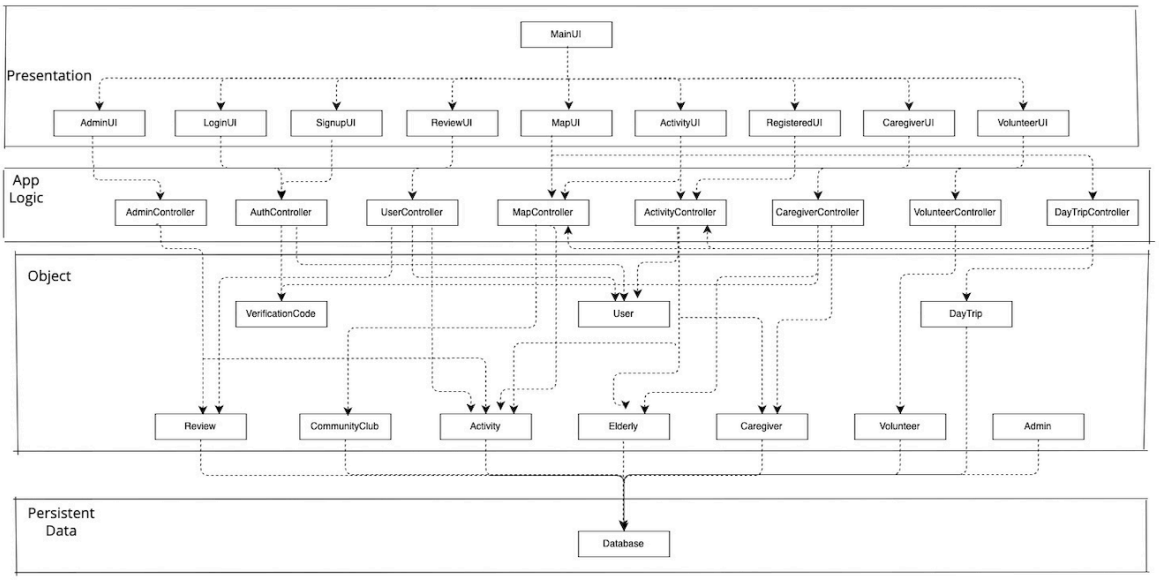
V.IV ForgotPassword



C. Dialog Map Diagram



3. System Architecture



A. Presentation Layer

The Presentation Layer is responsible for the interaction between users and the SilverConnect system. It includes different User Interfaces (UIs) for each role, and each UI communicates with the appropriate controller to carry out the required actions.

This layer consists of the following UIs:

1. AdminUI: AdminUI is part of the MainUI. It provides administrators with the tools to manage the system, such as verifying users, managing activities, and moderating reviews. The AdminController is called by AdminUI to handle these operations.

2. LoginUI: LoginUI is responsible for handling user login. It allows different types of users (e.g., Elderly, Volunteer, Caregiver, Admin) to log in with their credentials. The AuthController is used to verify login details and handle authentication processes.

3. SignupUI: SignupUI allows new users to create an account by providing personal details, role type, and password. The AuthController is responsible for storing user data in the system.

4. ReviewUI: ReviewUI enables users to submit, view, and edit reviews for activities they participate in. The ReviewController interacts with this UI to process review submissions.

5. MainUI: MainUI serves as the entry point and navigation hub for the entire system. It connects all sub-UIs and helps users navigate between authentication, activity listings, and registered activities. It coordinates with various controllers, such as UserController, AuthController, and ActivityController.

6. ActivityUI: ActivityUI displays all available activities for users. It allows users to browse, filter, and join activities. It works closely with the ActivityController to retrieve and manage activity data.

7. MapUI: MapUI displays an interactive map showing nearby community clubs and activity venues. It also provides directions for users to reach their activities. MapController helps retrieve the coordinates and directions.

8. RegisteredUI: RegisteredUI lists all activities that the user has registered for. Users can also cancel their registration through this UI. ActivityController manages the registration data.

9. CaregiverUI: CaregiverUI is designed for caregivers to manage elderly profiles. It allows caregivers to link and unlink elderly users and send verification codes to confirm the linkage. The CaregiverController is used to manage these actions.

10. VolunteerUI: VolunteerUI allows volunteers to browse and join activities, view join requests, and cancel them if necessary. It interacts with the VolunteerController to manage volunteer actions.

B. App Logic Layer

The App Logic Layer contains the controllers that facilitate communication between the Presentation Layer and the Object Layer. These controllers encapsulate the core business logic and provide an interface for the UIs to interact with.

This layer consists of:

1. AdminController: AdminController is responsible for administrative tasks, including verifying users, banning users, notifying users, and processing reviews. It interacts with the User, Activity, and Review entities.

2. AuthController: AuthController handles user authentication, including login, signup, and password verification. It communicates with User entities and is used by the LoginUI and SignupUI.

3. UserController: UserController manages user-related operations, such as retrieving and updating user profiles, and handling review submissions. It interacts with the User and Review entities and supports UIs like ReviewUI and CaregiverUI.

4. MapController: MapController provides location-based services, such as showing nearby community clubs and activities, and generating optimal routes. It works with the Activity and DayTrip entities and collaborates with the MapUI.

5. DayTripController: DayTripController generates tailored day trip recommendations based on users' interests and activities. It integrates data from the Activity, User, and Review entities to offer recommendations, which are shown through MapUI.

6. ActivityController: ActivityController manages all activity-related operations. It retrieves available activities, handles user registrations, and ensures that activity capacities are maintained. It interacts with the Activity and User entities.

7. CaregiverController: CaregiverController supports caregivers in linking and unlinking elderly profiles. When linking is successful, it updates the Elderly entity. It also manages caregiver-elderly relationships in the system.

8. VolunteerController: VolunteerController allows volunteers to browse, join, or cancel registrations for activities. It interacts with the User and Activity entities to handle volunteer-related actions.

C. App Object Layer

The Object Layer consists of entities representing the core business objects in the system. These entities interact with the controllers to implement the business logic, and their data is stored and retrieved from the Persistent Data Layer.

This layer consists of:

1. User: The User entity contains information such as user ID, name, password, email, role (Elderly, Caregiver, Volunteer, Admin), and other personal details. It is used by the AuthController and UserController.

2. Activity: The Activity entity contains details about community activities, such as activity ID, name, location, date, and capacity. It is used by the ActivityController to manage and retrieve activities.

3. Review: The Review entity stores feedback for activities, including text, photos, ratings, and the user who submitted the review. It is used by the ActivityController and ReviewUI to display and submit reviews.

4. Elderly: The Elderly entity represents elderly users in the system, including their ID, name, age, and caregiver relationships. It is updated by the CaregiverController.

5. Caregiver: The Caregiver entity represents caregivers, storing their ID and the elderly profiles they manage. It is updated by the CaregiverController.

6. Volunteer: The Volunteer entity represents volunteers and the activities they are registered for. It is used by the VolunteerController to manage volunteer actions.

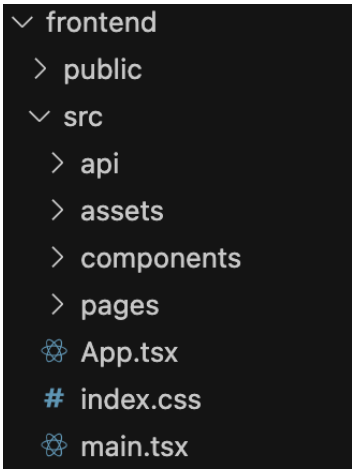
7. DayTrip: The DayTrip entity contains recommendations for day trips tailored to elderly users. It integrates activities, hawker locations, and parks. It is created by the DayTripController.

D. Persistent Data Layer

The Persistent Data Layer consists of the database that stores all the entities and maintains the state of the system. This includes users, activities, reviews, and caregiver relationships, among other critical data. The database enables data persistence, retrieval, and modification as required by the controllers.

4. Application Skeleton

A. Frontend (React.js / TypeScript / Tailwind CSS)



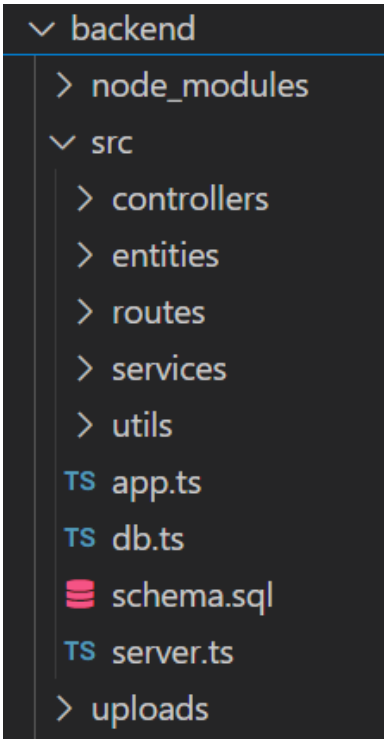
The frontend of SilverConnect is developed using React.js and TypeScript, providing a modular and maintainable architecture. It consists of user interfaces organized by user roles (Elderly, Caregiver, Volunteer, Admin) and functionality.

- **main.tsx** – Initializes the React app and mounts it to the DOM.
- **App.tsx** – Defines client-side routing (React Router) to all pages.
- **pages/** – Contains all UI screens for the site (e.g., activity listing and details, registration flows, admin views), organized by role and functionality.
- **components/** – Reusable UI elements shared across pages(e.g. Navbar.tsx).
- **api/** – Frontend HTTP clients for calling backend endpoints.
- **public/** – Static assets (images, icons, backgrounds like park.jpg).
- **index.css** – Initializes Tailwind CSS and imports global styles, including Leaflet's map styles. It also defines custom animations such as fade-in-out used for UI transitions and toast notifications.

Styling: The frontend uses Tailwind CSS for styling, offering a utility-first approach that enables rapid UI development directly within TSX. This ensures responsive layouts, a modern design system, and consistent styling without maintaining large external CSS files.

This structure ensures a clean separation of concerns, promotes code reuse, and makes it easier to manage updates across different parts of the application.

B. Backend (Node.js / Express / TypeScript)



The backend of SilverConnect is implemented using Node.js, Express, and TypeScript. It is responsible for handling all API logic, managing database interactions, and serving data to the frontend. The backend follows a modular folder structure for better organization and maintainability.

- **controllers/** – Contains all controller files that define the core logic for various backend services. Each controller processes incoming requests, calls the relevant service functions, and sends structured responses to the frontend. For example, user.controller.ts, activity.controller.ts, and auth.controller.ts handle user management, activity registration, and authentication respectively.

- **entities/** – Contains the definitions of data models or entities that represent key objects in the application (e.g., users, volunteers, caregivers, activities). These definitions help ensure consistency when performing CRUD operations.

- **routes/** – Defines the API endpoints that connect the frontend to the backend. Each route maps an HTTP request (GET, POST, PUT, DELETE) to the corresponding controller function. For instance, /api/users, /api/activities, /api/auth.

- **services/** – Handles external and business logic operations, such as calling government APIs or other third-party services to update or verify application information. It separates external data-handling logic from controllers for cleaner code organization.

- **utils/** – Contains utility functions and helper modules (e.g., authentication middleware, validation helpers, token generation) that are reused across different parts of the backend.

- **db.ts** – Responsible for initializing and managing the application's database connection. It defines a flexible database interface that supports both PostgreSQL and MySQL, allowing smooth interaction between the application and the database.

- **schema.sql** – Contains the SQL schema used to initialize the database structure, including table definitions for users, activities, volunteers, caregivers, and relationships.

- **app.ts** – Configures the Express application, including middleware setup (e.g., JSON parsing, CORS), and registers all API routes before the server starts.

- **server.ts** – The entry point of the backend that starts the Express server and listens for incoming client requests. It ensures the backend is properly initialized and ready to handle API traffic.

- **uploads/** – A directory used to store uploaded images and files (e.g., user profile pictures or activity images) that are referenced within the app.

5. Appendix

A. **Key Design Issues**

I. Identifying and Storing Persistent Data

- Database
  - Users (Elderly, Caregiver, Volunteer, Admin)
    - name, email, password, role, age, contact_number, preferred_language
      - Volunteers
        - user_id, availability, preferred_activities, location_radius
      - Caregiver
        - elderly_id, caregiver_id, relationship
      - Elderly
        - user_id
      - Admin
        - user_id, pa_email
  - Activities
    - name, description, date, start_time, end_time, capacity, vacancies, cost, location, status, community_club_id, image_url
  - Community_clubs
    - Name, address, postal_code
  - Volunteer_Join_Requests
    - Volunteer_id, activity_id, elderly_id, status, message

II. Providing Access Control

- Encryption
  - Encrypt passwords to be stored in database
- Access Control Models: Role-Based
  - Admin vs Users, admin can edit, cancel and change activities, users can only join.

B. Tech Stack

- **Frontend**
  - React.js
  - TailwindCSS
- **Backend**
  - Node.js
  - Express
- **Code**
  - TypeScript
- **Database**
  - PostgreSQL